

Optimización del Problema Climbing Stairs

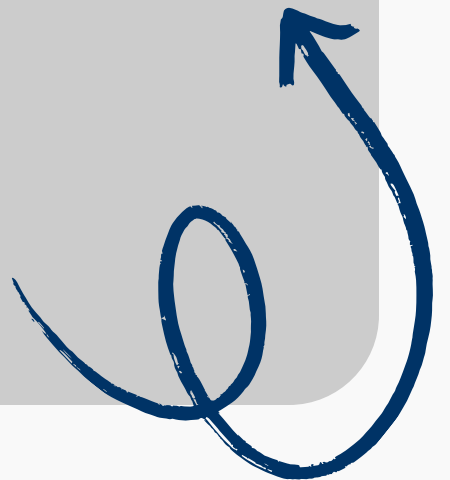
Jhon Perlaza, Heidi Sofía
Barrera, Andrés Felipe Cadena,
Juan Ahumada



El Problema de la Escalera

Entendiendo las posibilidades

Este problema se centra en encontrar cuántas combinaciones diferentes de pasos se pueden tomar para alcanzar el escalón n , subiendo 1 o 2 escalones a la vez.



Relación con Fibonacci

Conexiones entre escalones y secuencias

Estructura Recursiva

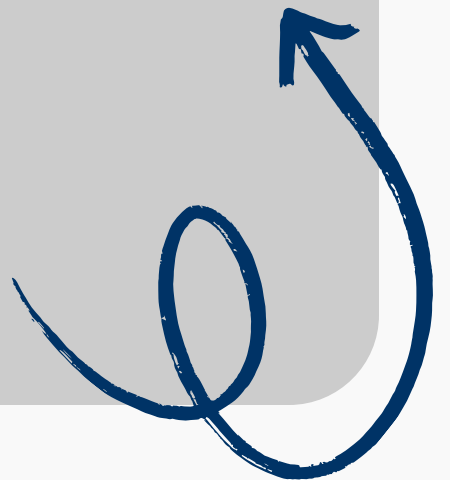
La relación de W de n con Fibonacci establece una **estructura recursiva** similar, donde cada número se forma a partir de la suma de los dos anteriores, creando un patrón.

Inducción Matemática

Se puede demostrar mediante **inducción matemática** que W de n es equivalente a F de n más 1, ampliando nuestra comprensión de la secuencia a través de la combinatoria.

Enfoques Alternativos

La conexión entre estos conceptos ofrece **enfoques alternativos** para resolver problemas de combinatoria: la recursión ingenua y la fórmula cerrada, cada una con sus propias implicaciones de complejidad.



Implementación Pesada

Recursión Ingenua Exponencial en Profundidad

Método Recursivo

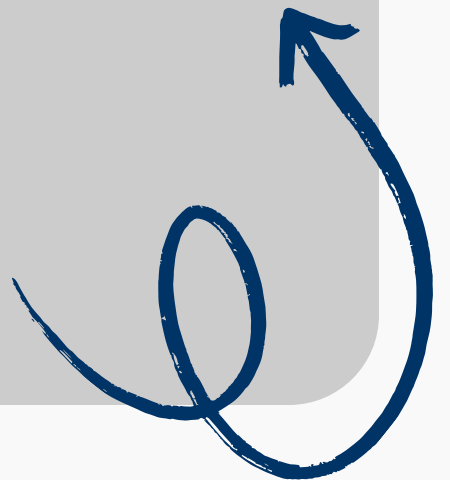
La implementación recursiva **invoca** la función W de n directamente, llamando a W de n menos 1 y W de n menos 2, lo que genera múltiples subproblemas.

Complejidad Exponencial

La complejidad de esta técnica se define como T de n es $\Theta(\phi^n)$, donde ϕ es aproximadamente 1.618, lo que resulta en un crecimiento explosivo de llamadas.

Repetición de Cálculos

Los subproblemas, como W de 3, se recalculan numerosas veces dentro de otras funciones, causando una ineficiencia significativa cuando n es elevado.



Implementación Optimizada

La Fórmula Cerrada de Binet

Fórmula cerrada

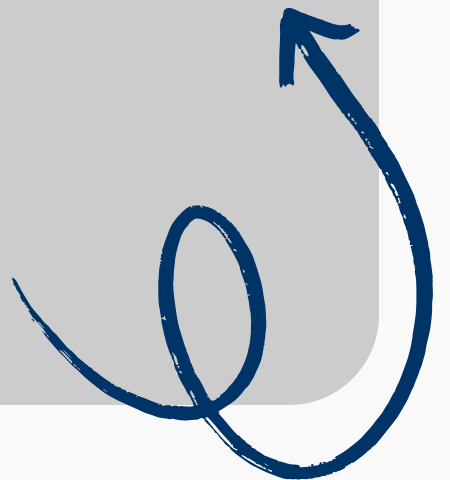
La fórmula cerrada F de k permite calcular W de n de forma directa. Esto reduce significativamente el tiempo de cómputo al evitar la recursión.

Estrategia híbrida

La implementación utiliza un enfoque híbrido: para n menor o igual a 70, se usa cálculo rápido; para n mayor a 70, se emplea precisión decimal.

Complejidad mejorada

Al aplicar esta fórmula, la complejidad algorítmica se reduce a $O(1)$, lo que permite resolver problemas de gran escala sin un crecimiento explosivo en el tiempo de ejecución.



Análisis del rendimiento

Comprendiendo la lentitud inicial

Costo constante

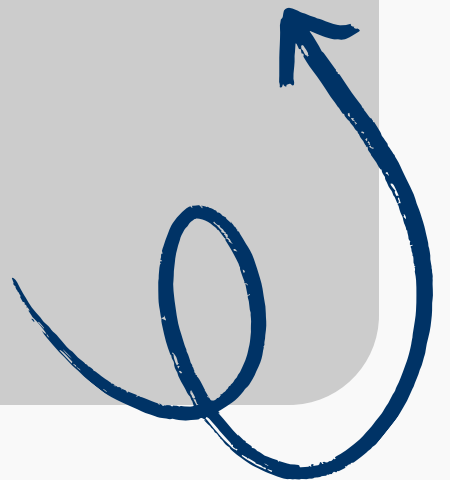
La evaluación de potencias y la preparación de decimales introduce un **costo fijo** constante que puede afectar el rendimiento para valores pequeños de n .

Ruido de medición

Las variaciones en el tiempo de ejecución pueden ser influenciadas por factores externos, como **el sistema operativo** y el ambiente de ejecución de Python, introduciendo ruido en las mediciones.

Analogía de comparación

Comparar una pequeña distancia, como caminar 2 metros, con una distancia larga, como conducir 100 km, ilustra cómo la eficiencia puede **cambiar radicalmente** a medida que n crece.



El Benchmark

Configuración y Código de Prueba

Valores probados

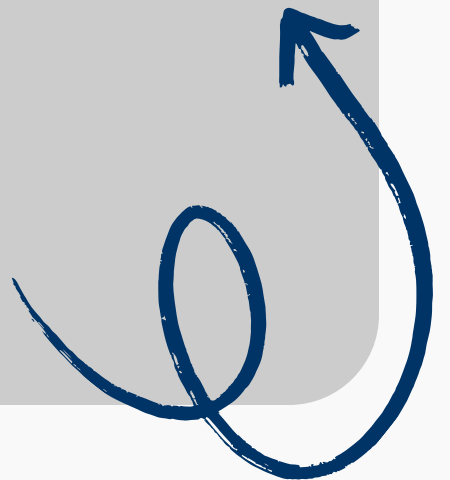
Se evaluaron múltiples valores de n , incluyendo 1, 2, 3, 5, 10, 15, 20 y más, para captar el rendimiento en una variedad de escenarios prácticos.

Repeticiones para precisión

Se realizaron siete repeticiones por cada medición, promediando los resultados para asegurar la fiabilidad de los datos y minimizar el impacto del ruido de medición.

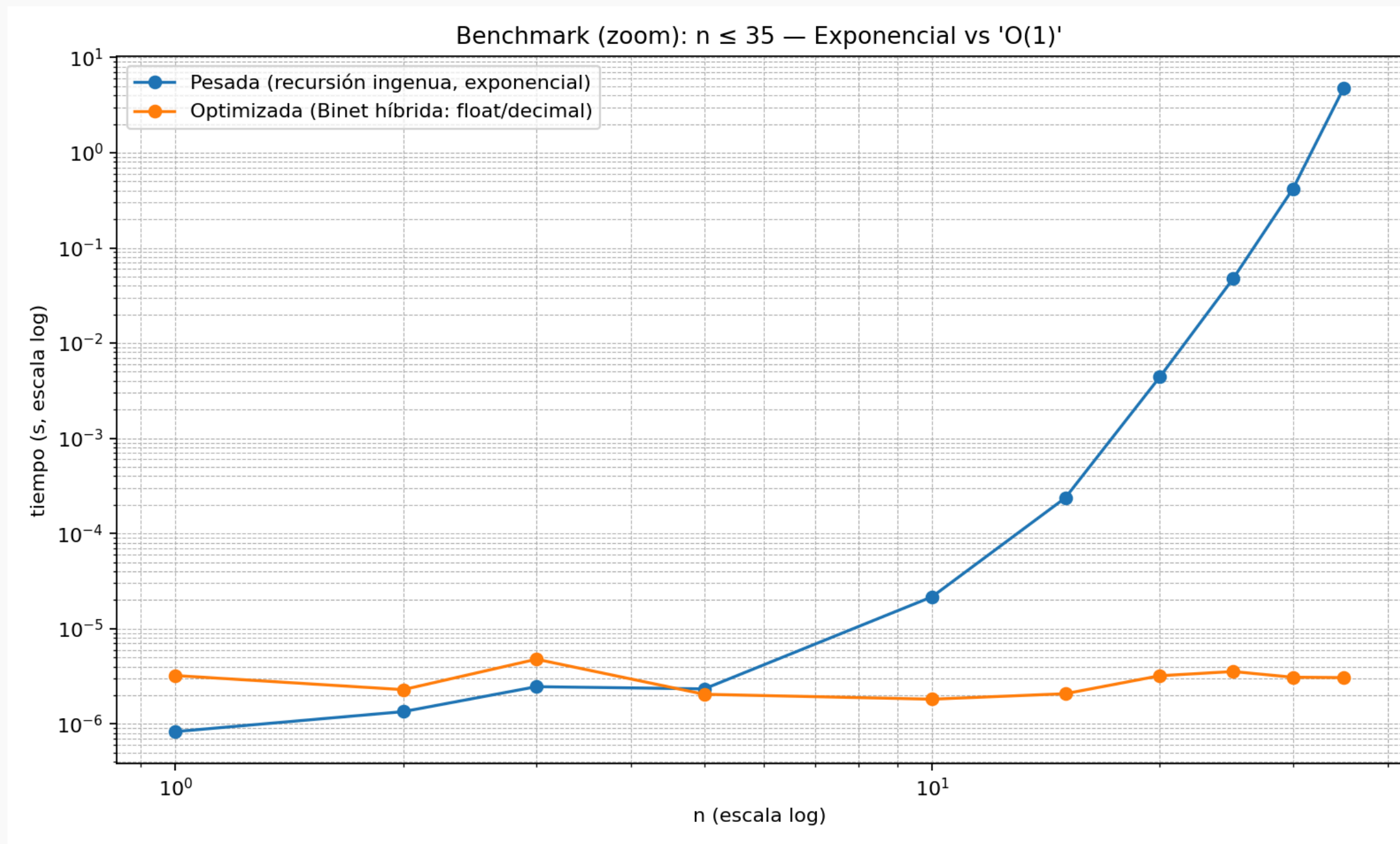
Código del benchmark

El código fue diseñado para comparar la recursión ingenua y la optimizada, asegurando que ambas implementaciones devolviesen los mismos resultados para la validación del rendimiento.



Gráfica de Rendimiento Comparativo

Este gráfico muestra la **comparación de rendimiento** entre la implementación ingenua y la fórmula de Binet para valores de n menores o iguales a 35, destacando la eficiencia de la optimización.



Comparación de Escalabilidad para n Masivos

Este gráfico ilustra claramente cómo el **algoritmo optimizado** supera al exponencial a medida que n se incrementa. A partir de n igual a 35, el rendimiento del algoritmo pesado se vuelve impracticable, mientras que el optimizado sigue siendo eficiente.

